Training Small Language Models under a Fixed Compute Budget

Group Composition:

1. Wong Chuan Kai, Royston, A0271912H, e1121315@u.nus.edu
2. Chan Wei Wen Kevin, A0272202W, e1121605@u.nus.edu

Github Repository: https://github.com/Alutare/DSA4212_Group_10_Assignment

Abstract

We investigate the trade-off between model capacity and training exposure under a fixed time budget. Across 36 training runs, the medium (M) configuration achieves the best observed balance: its main size-study run reaches 1.1461 test BPB with 17.01 million parameters, compared with 1.2710 for the 97.73-million-parameter model. Three repeats of M average 1.1544 ± 0.0031 test BPB, and every M repeat beats every repeat of the large (L) model. BPE16K gives the lowest observed BPB among five tokenizers, but one run per tokenizer limits conclusions about the robustness of this ranking. Learning-rate and batch experiments indicate that neither greater capacity nor higher throughput alone ensures better performance. Our conclusions concern the tested configurations and budget, not a universal optimal model size.

Budget. Each final neural-model run receives 1,800 seconds of measured optimisation-step time on one NVIDIA Tesla T4 GPU in Google Colab. Preliminary learning-rate runs receive 300 seconds. Validation, checkpoint writing, and preprocessing are excluded from this per-run allowance.

Initialisation and objective. All neural weights and embeddings are initialised from scratch. We compare next-token prediction on WikiText-103 raw using validation bits per byte (BPB) for selection and held-out test BPB for final assessment. Existing GPT-2 tokenisation is used without pretrained neural weights.

1 Introduction and Related Work

Our central question is: *Under a fixed GPU training-time budget, can a smaller Transformer trained on more tokens outperform a larger Transformer trained on fewer tokens?* Larger models offer greater capacity but cost more per token; very small models train quickly but may lack sufficient capacity. The objective is to allocate limited training time effectively.

Hoffmann et al. [1] show that model size and training tokens should be considered jointly when allocating computation. Their large-scale results motivate our question, but we do not directly test their scaling law: our models, dataset, and budget are smaller, and equal time does not imply equal FLOPs. Muennighoff et al. [3] study repeated data under constrained data availability. This motivates distinguishing token exposure from unique text in our experiments, rather than treating every processed token as new information.

We study model size as the main factor, supported by tokenizer comparisons, learning-rate tuning, batch sensitivity, and repeated runs. We report unsuccessful settings and limitations alongside the best observed configuration.

2 Methods and Experimental Design

2.1 Data and model

We use the official training, validation, and test splits of WikiText-103 raw (`wikitext-103-raw-v1`), from the WikiText corpus [2]. Custom byte-level BPE tokenizers are fitted on training text only. Each non-null text row is encoded and followed by an encoded newline. UTF-8 byte counts follow the same convention across tokenizers. Token streams are stored as memory-mapped arrays.

Training samples random contiguous windows. Reported equivalent epochs are tokens processed divided by the training stream's token count; they are not guaranteed complete passes through every example. The BPE16K training stream contains approximately 123.79 million tokens.

Our nanoGPT-inspired decoder-only Transformer [4] uses causal self-attention, pre-layer normalisation, GELU feed-forward layers of width $4d$, learned positional embeddings, and tied input/output token embeddings. Context length is 256 tokens and dropout is zero. Linear and embedding weights use Gaussian initialisation with standard deviation 0.02; residual projections use $0.02/\sqrt{2L}$ for L layers. No pretrained neural components are loaded.

Table 1: BPE16K model configurations. Parameter counts are in millions; non-embedding counts exclude token and positional embeddings. LR is the peak rate used in final size runs.

Model	Layers	Heads	Width	Total	Non-emb.	LR
XS	4	4	128	2.92	0.79	0.0032
S	4	4	256	7.41	3.15	0.0032
M	6	6	384	17.01	10.62	0.0032
L	8	8	512	33.69	25.17	0.0024
XL	12	12	768	97.73	84.95	0.0016

2.2 Optimisation and budget accounting

Training uses AdamW with $(\beta_1, \beta_2) = (0.9, 0.95)$, weight decay 0.1 on matrix parameters, gradient clipping at norm 1.0, and FP16 mixed precision with gradient scaling. Learning rate warms up over 100 training steps, then follows a cosine schedule driven by the fraction of training time consumed, with a minimum of 10% of the peak rate.

The baseline effective batch is 16,384 tokens per step. XS through L use 32 sequences per micro-batch and two accumulation steps; XL uses 16 sequences and four accumulation steps. Timing includes data fetching, forward/backward computation, and the optimiser step, with GPU synchronisation before recording elapsed time. It excludes validation and checkpoint writing. The last step may slightly exceed the allowance; checkpoint resumption preserves time already consumed. Logged steps and tokens count training iterations and processed examples; they do not separately count any optimiser updates skipped by FP16 gradient scaling.

The 17 tuning and 19 final runs have a nominal combined allowance of $17(300) + 19(1800) = 39{,}300$ seconds, or 10.92 GPU-hours. This is not the total project cost: preprocessing, tokenizer fitting, smoke tests, benchmarks, evaluation, and storage overhead add time.

2.3 Evaluation

For scored token positions $\mathcal{S}$, let

$$\mathcal{L} = -\sum_{t \in \mathcal{S}} \log p_\theta(x_t \mid \text{available preceding context}).$$

The implementation reports

$$\widehat{\mathrm{BPB}} = \frac{\mathcal{L}}{n \ln 2} \frac{N}{B},$$

where $n = |\mathcal{S}|$, N is the split's token count, and B its UTF-8 byte count. Full evaluation scores all but the first token, so $n = N - 1$. Thus the implementation estimates byte-normalised loss from mean token loss; it differs from $\mathcal{L}/(B \ln 2)$ by the negligible factor $N/(N-1)$ on full splits.

BPB makes scores comparable in byte units across vocabularies, unlike token perplexity. It does not equalise byte context: 256 tokens represent different amounts of text for different tokenizers. Evaluation uses stride 256 and masks already-scored positions in the final right-aligned window. Periodic validation uses 64 fixed windows; final validation and test use full splits. Final checkpoints are evaluated, rather than choosing the minimum of the partial-validation curve.

2.4 Study design and baselines

Table 2 lists completed runs, excluding optional notebook studies without saved results. Validation determines configuration selection; final neural test evaluation covers five tokenizer, five size, and six repeated-run checkpoints. Batch sensitivity remains validation-only. Baseline test scores are also computed in the notebook before the final neural evaluations in Section 18.

The tokenizer study fixes the backbone, peak learning rate, token context, effective token batch, and time allowance, but vocabulary-related parameters vary. The size study changes depth, width, and learning rate jointly. It compares practical configurations, not the isolated causal effect of parameter count. XS, M, and XL receive direct learning-rate sweeps; S and L rates are interpolated. Train-fitted unigram and interpolated bigram models provide predictive baselines, with bigram interpolation selected on validation data.

Table 2: Completed training studies.

Study	Comparison	Runs	Seconds/run
Learning rate	XS, M, XL; grid and two extensions	17	300
Tokenizer	GPT-2, BPE4K/8K/16K/32K with M	5	1,800
Model size	XS, S, M, L, XL with BPE16K	5	1,800
Batch/LR	Three XL settings	3	1,800
Repeats	M and L, three seeds each	6	1,800

3 Results and Analysis

3.1 Intermediate capacity performs best

M achieves the lowest BPB in the main size study (Table 3). It uses 17.4% of XL's parameters and processes 5.03 times as many tokens. Its test BPB is lower by about 0.125 (9.8%). Its advantage over L is smaller, at 0.0103 BPB.

Table 3: Size study with BPE16K, 1,800 seconds per run. Exposure denotes equivalent epochs; lower BPB is better.

Model	Tokens (M)	Exposure	Val BPB	Test BPB
XS	491.36	3.97	1.2907	1.3102
S	346.57	2.80	1.1862	1.2068
M	187.43	1.51	**1.1261**	**1.1461**
L	97.17	0.78	1.1375	1.1564
XL	37.26	0.30	1.2530	1.2710

Figure 1 plots validation BPB (Table 3 gives both validation and test values) and visualises the non-monotonic size–performance relationship and the steep reduction in token exposure as size increases. Figure 2 shows how validation BPB changes over training time and token exposure; its intermediate fixed-window measurements should not be read as full-split final evaluations.

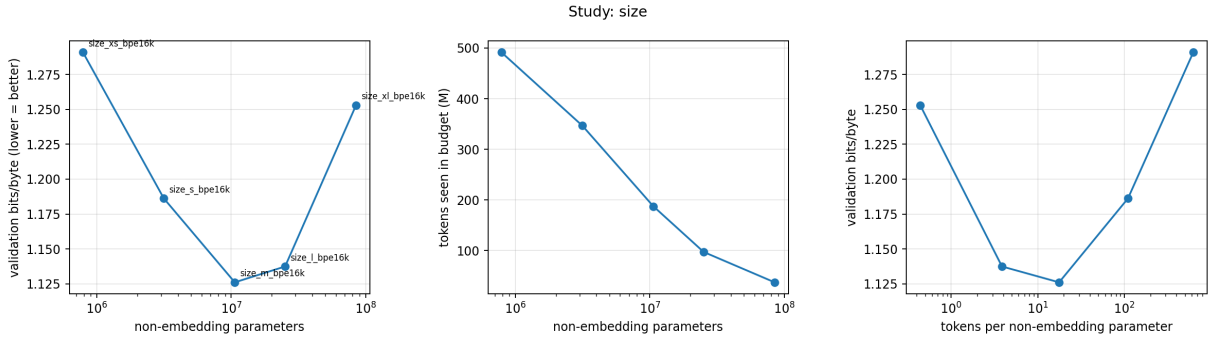


Figure 1: Main size study: validation BPB versus non-embedding parameter count, tokens processed under the fixed time budget, and BPB versus tokens per non-embedding parameter. Lower BPB is better. The five points are single runs, not seed averages.

XS processes the most tokens yet performs worst in this comparison. Maximising throughput alone is therefore insufficient. The pattern is consistent with limited capacity for small models and insufficient

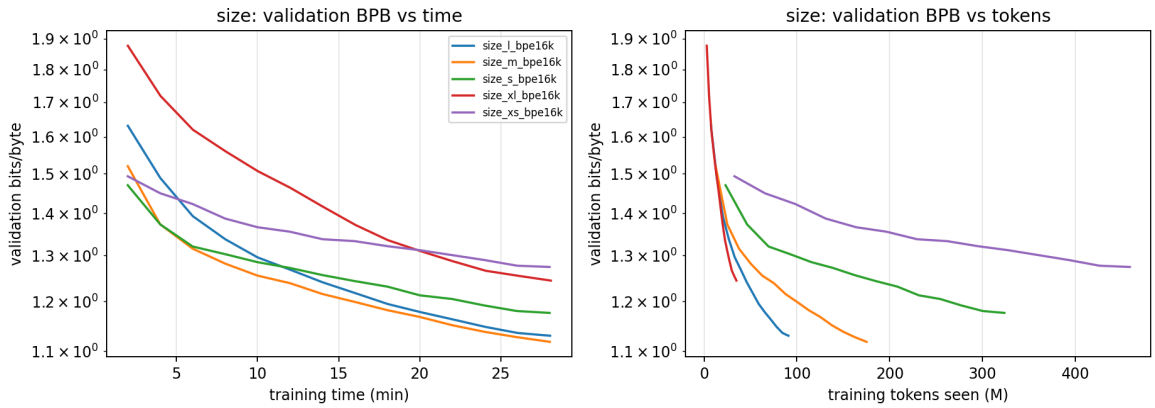


Figure 2: Periodic validation BPB versus optimisation time (left) and cumulative training tokens (right) for the five model sizes. These intermediate values are based on a fixed subset of validation windows; final scores in Table 3 use the full split.

training exposure for XL, but neither mechanism is isolated experimentally. Longer-budget runs would be needed to determine whether XL eventually overtakes M.

The estimated training computation is 9.28 PFLOPs for XS, 20.35 for M, and 22.86 for XL. Model FLOPs Utilisation (MFU), calculated relative to the T4's assumed peak FP16 throughput of 65 TFLOPs/s, is 7.9%, 17.4%, and 19.5%, respectively. Both FLOPs and MFU are model-based estimates rather than direct hardware measurements. Despite greater estimated computation and MFU, XL achieves worse test BPB than M, showing that higher computational expenditure does not necessarily improve predictive performance under a fixed time budget.

3.2 Vocabulary size trades sequence length against cost

BPE16K has the lowest validation and test BPB in Table 4. It is the best observed tokenizer for this backbone and budget; only one run per tokenizer limits conclusions about small differences.

Table 4: Tokenizer study with the M backbone. Bytes/token is measured on training text; token counts include repeated exposure.

Tokenizer	Bytes/token	Tokens (M)	Val BPB	Test BPB
GPT-2	4.520	92.13	1.1793	1.2038
BPE4K	3.494	261.01	1.1760	1.1921
BPE8K	3.964	233.37	1.1443	1.1627
BPE16K	4.371	192.43	**1.1222**	**1.1447**
BPE32K	4.678	133.89	1.1290	1.1530

Figure 3 illustrates the competing effects of vocabulary size on predictive quality, throughput, and the model parameter allocation.

Moving from BPE16K to BPE32K doubles the tied token embedding matrix from 6.29M to 12.58M parameters, but bytes/token rises only about 7%. Processed tokens fall by approximately 30%, and test BPB worsens by 0.0083. The added vocabulary cost is not offset by shorter sequences in this experiment. BPE4K processes

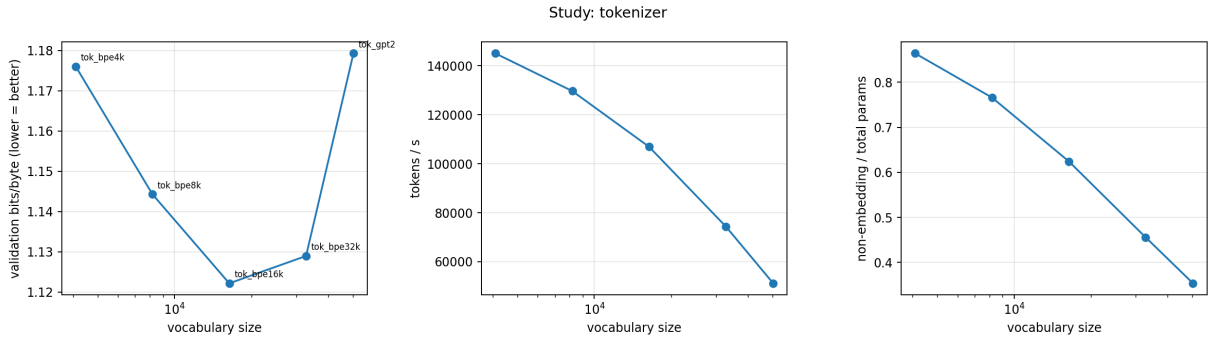


Figure 3: Tokenizer study: validation BPB versus vocabulary size, training throughput, and the fraction of model parameters outside token and positional embeddings. The M backbone is fixed, while vocabulary-dependent total parameters vary.

more tokens but represents less text per token and also performs worse.

The BPE16K tokenizer run, the M size run, and the three M repeats are separate executions of the same nominal configuration. Their test BPB values are 1.1447, 1.1461, and 1.1544 ± 0.0031 respectively. This variation highlights the importance of execution differences and token exposure when interpreting small gaps between configurations. In particular, the 0.0083 BPB gap between BPE16K and BPE32K is smaller than the difference between the main M run and the repeated-run mean. Although BPE16K has the best observed score, the single-run tokenizer comparison does not establish that its small advantage over nearby vocabulary sizes is statistically robust. Some tokenizer runs were resumed and used different recorded code revisions, so configuration equality alone does not guarantee identical execution conditions.

GPT-2 differs in vocabulary size, learned merges, and tokenisation design; its result cannot isolate an in-domain vocabulary benefit. A custom 50K tokenizer was prepared, but no corresponding completed model comparison is recorded. Even matching vocabulary size would not by itself control every tokenisation difference.

3.3 Learning-rate tuning qualifies the size ranking

The initial sweep tests $\{0.0002, 0.0004, 0.0008, 0.0016, 0.0032\}$ for XS, M, and XL. Two additional runs test 0.0064 for XS and M. For M, increasing the rate from 0.0032 to 0.0064 worsens five-minute validation BPB from 1.3165 to 1.6918. The recorded loss remains finite: this is poor performance, not recorded numerical divergence. XS instead improves from 1.3934 to 1.3628.

Nevertheless, the final XS size run uses 0.0032. M is therefore the best tested configuration, not an exhaustive optimum over sizes and hyperparameters. Five-minute tuning may also favour settings that are suboptimal at 30 minutes. The extensions used different token exposures and code revisions, further limiting a purely causal learning-rate interpretation.

3.4 Batch sensitivity: more tokens need not mean better learning

The XL study uses $\eta(b) = 0.0016\sqrt{b/16{,}384}$ for effective batch b. Table 5 therefore tests a joint batch/LR policy, not batch size alone.

Table 5: XL sensitivity under the same time allowance. Steps are logged training iterations.

Tokens/step	Peak LR	Steps	Tokens (M)	Val BPB
4,096	0.0008	8,589	35.18	1.3119
16,384	0.0016	2,242	36.73	**1.2505**
65,536	0.0032	575	37.68	1.4633

The largest batch processes 2.6% more tokens than the baseline but completes approximately 74% fewer steps and performs substantially worse. The smallest batch completes more steps but also worsens performance. Thus neither more steps nor more tokens alone explains learning quality.

The common 100-step warm-up covers 17.4% of the largest-batch run's steps versus 4.5% for the baseline. Batch size, learning rate, step count, and warm-up allocation interact. The tested smaller-batch policy does not rescue XL, but this does not rule out a better independently tuned policy. A follow-up should control learning rate and use time-proportional warm-up.

3.5 Repeated runs preserve the M–L ranking

Table 6 reports means and sample standard deviations separately for seeds 1337, 1338, and 1339. Every M repeat has lower test BPB than every L repeat; the mean gap is 0.00942 BPB. This supports the main ranking but is not a large replication study or a formal significance claim.

Table 6: Three repeated runs per model; mean $\pm$ sample standard deviation.

Model	Validation BPB	Test BPB
M	1.13418 ± 0.00181	1.15438 ± 0.00310
L	1.14385 ± 0.00062	1.16380 ± 0.00164

M processes 152.57–163.99M tokens in the repeated runs, compared with 187.43M in the original size run. The lower token exposure may contribute to the higher average BPB of the repeated runs. Even runs using the same seed differ in exposure. These standard deviations therefore combine seed and execution variability and should not be interpreted as pure initialisation noise.

3.6 Non-neural references

On test data, the BPE4K unigram obtains 2.8483 BPB and the interpolated bigram 1.9267, compared with 1.1921 for the M Transformer using BPE4K. This establishes improvement over simple frequency-based references under shared tokenisation, without attributing the improvement solely to context length. These models are not matched in capacity or training cost. The bzip2 reference obtains 2.2063 bits per byte but adapts to the evaluated text, so it is not an equivalent train-fitted held-out language model.

4 Limitations and Reproducibility

Our evidence covers one corpus, GPU type, context length, and final time budget. Depth and width change jointly, learning-rate tuning is incomplete, and tokenizer comparisons change byte context and vocabulary cost. Equivalent epochs measure repeated exposure, not unique data; the independent effect of repetition is untested. BPB does not measure reasoning, instruction following, or factual accuracy. Higher test than validation BPB alone does not diagnose overfitting because the splits contain different text.

The numerical evidence is in `results/all_experiments.csv`, `results/final_test.csv`, and per-run JSON records and logs under `results/runs/`. All 16 final neural evaluations are present in the current CSV. The implementation is under `code/`, and the repository workflow is `notebooks/DSA4212_Merged_v2.ipynb`. Run records retain arguments and code revisions; these should govern reproduction rather than later defaults.

Reproduction requires the original data splits, identical preprocessing and tokenizers, a compatible environment, and either saved checkpoints or retraining. Saved run arguments and code revisions define the reported configurations. In particular, the final XS run used a learning rate of 0.0032; repeating selection over the expanded tuning results, including the later 0.0064 runs, would change this choice. Checkpoints and prepared datasets are stored outside GitHub, so reproducing checkpoint-level evaluations requires access to those files or retraining.

The most informative follow-ups are longer-budget M/L/XL comparisons, a 30-minute XS run at the additional tuning candidate, and independently controlled batch/LR schedules. These are proposed experiments, not completed evidence.

5 Conclusion

Across the tested five sizes, the size–performance relationship is non-monotonic: increasing capacity from XS to M improves held-out BPB, while moving from M to L and XL worsens it under the same time allowance. The smallest models see more tokens but appear capacity-limited; the largest model has greater capacity but receives much less training exposure. Neither mechanism is isolated by this experiment. The M configuration with BPE16K offers the best observed balance between capacity and training exposure in our 30-minute T4 study. Its main size run reaches 1.1461 test BPB (repeat mean 1.1544 ± 0.0031), and every repeat preserves its advantage over L. The result supports joint consideration of capacity, tokenisation, and optimisation. It does not establish a universal optimum or identify one isolated cause of the ranking.

AI Assistance

AI tools were used to assist with code development, debugging, report drafting, language refinement, and numerical cross-checking. All experiments were executed by the group, and the reported results were obtained from our own training and evaluation runs. The group reviewed the generated suggestions, verified the experimental findings against the recorded outputs, and retained responsibility for the final implementation, analysis, and conclusions.

References

[1] J. Hoffmann et al. *Training Compute-Optimal Large Language Models*. 2022. https://arxiv.org/abs/2203.15556.

[2] S. Merity, C. Xiong, J. Bradbury, and R. Socher. *Pointer Sentinel Mixture Models*. 2016. https://arxiv.org/abs/1609.07843.

[3] N. Muennighoff et al. *Scaling Data-Constrained Language Models*. 2023. https://arxiv.org/abs/2305.16264.

[4] A. Karpathy. *nanoGPT*. Software repository. https://github.com/karpathy/nanoGPT.